

# FaceAuth — Documentation Officielle

SDK Web d'authentification biométrique avec détection de vivacité  
(Liveness Detection) — Spécifications Techniques, Business et  
Conformité

Next

Juin 2026 — Version 1.0

## Introduction et vision du produit

- Présentation générale

- Public cible

- Cas d'usage couverts

- Roadmap des SDKs

## Architecture technique

- Stack technique du MVP

- Paradigme de sécurité hybride client/serveur

  - Pourquoi cette séparation est nécessaire

- Les quatre gestes de sécurité (liveness)

- Optimisation du backend FastAPI et DeepFace

  - Pré-chargement des modèles en mémoire

  - Normalisation des images en amont

  - Gestion de la concurrence

- Optimisation de la base de données (PostgreSQL + pgvector)

  - Indexation vectorielle HNSW

  - Stratégie de cache Redis

- Infrastructure et scaling (Railway)

  - Point d'attention : cold start

- Workflow chronométré d'une vérification réussie

## Besoins fonctionnels et non fonctionnels

- Besoins fonctionnels

  - Dashboard développeur

  - Dashboard client / module de scan

- Besoins non fonctionnels

  - Performance et latence

  - Sécurité des données

  - Fiabilité — marge de sécurité

  - Clarté UX

## Spécifications de l'interface utilisateur (UX)

- Principe directeur : zéro redirection

- Comportement adaptatif (desktop vs mobile)

  - Sur ordinateur (desktop) — modale centrée

  - Sur smartphone (mobile) — bottom-sheet immersive

- Règles de sécurité UX et ergonomie

- Comportement du thème

  - Mode clair (détection automatique)

  - Mode sombre (détection automatique)

  - Fallback système

## Approfondissement — Sécurité anti-spoofing

- Le problème : une liveness purement côté client est falsifiable

- Recommandations pour le MVP

  - 1. Jeton de session de vérification (Challenge Token)

  - 2. Preuve d'exécution côté client (Attestation légère)

3. Vérifications anti-spoofing serveur (passives)

4. Roadmap post-MVP

Synthèse

Approfondissement — Modèle économique et tarification

Rappel du modèle actuel

Analyse de la structure tarifaire

Cohérence de la dégressivité

Estimation des coûts d'infrastructure par vérification

Le pack "Découverte" gratuit — opportunités et risques

Positionnement concurrentiel

Pack Entreprise

Conformité réglementaire et protection des données (KYC)

Contexte

Principes déjà intégrés dans la conception technique

Points à formaliser pour une documentation de conformité robuste

1. Statut juridique de FaceAuth (sous-traitant vs responsable de traitement)
2. Base légale du traitement
3. Registre des traitements et durée de conservation des embeddings
4. Localisation des données et transferts internationaux
5. Journalisation et auditabilité
6. Sécurité technique attendue
7. Biais algorithmique et équité

Tableau de synthèse — feuille de route conformité

Avertissement

Synthèse et prochaines étapes

Récapitulatif de faisabilité

Prochaines étapes suggérées

# Introduction et vision du produit

---

## Présentation générale

FaceAuth est un SDK d'authentification biométrique clé en main, conçu pour permettre aux développeurs d'intégrer une couche de sécurité faciale robuste, assortie d'une détection de vivacité (Liveness Detection) basée sur des défis gestuels aléatoires. Le SDK est pensé comme un service à la demande ("Biometric as a Service"), facturé à l'usage, sans engagement initial.

## Public cible

### Startups Fintech (Afrique et International)

FaceAuth répond à un besoin critique pour les acteurs financiers : sécuriser le KYC (Know Your Customer), l'enrôlement des nouveaux clients, et la validation des transactions sensibles, dans des contextes où la fraude identitaire est un risque majeur.

### Plateformes SaaS et E-commerce

Pour ces acteurs, FaceAuth permet d'éliminer la quasi-totalité des inscriptions automatisées par des robots (bots), tout en proposant une expérience utilisateur moderne, fluide et différenciante par rapport aux CAPTCHA traditionnels.

## Cas d'usage couverts

FaceAuth est conçu comme un SDK de vérification d'identité destiné aux actions sensibles d'une application. Les cas d'usage typiques incluent :

- Récupération de mot de passe oublié
- Connexion détectée comme à risque (nouvel appareil, nouvelle localisation)
- Validation de transaction financière
- Modification de coordonnées bancaires
- Changement d'adresse e-mail
- Suppression de compte
- Signature électronique

En complément de l'authentification proprement dite, le système de liveness de FaceAuth constitue une **défense efficace contre les bots** : il permet d'empêcher les créations massives de comptes automatisées, de limiter certaines formes de fraude

automatisée, et de garantir la présence d'un être humain réel derrière chaque action critique.

## Roadmap des SDKs

Le développement suit une logique progressive, en commençant par le web avant d'étendre vers le natif et le cross-platform :

1. **SDK Web** (priorité MVP) — JavaScript / TypeScript
2. **SDK natifs** — Swift (iOS) et Kotlin (Android)
3. **SDK cross-platform** — Flutter et React Native

# Architecture technique

---

## Stack technique du MVP

| Composant                         | Technologie retenue                        | Rôle                                                     |
|-----------------------------------|--------------------------------------------|----------------------------------------------------------|
| Frontend (Dashboards & Scan)      | React                                      | Interfaces développeur et utilisateur final              |
| Moteur IA Backend                 | FastAPI (Python), hébergé sur Railway      | Traitement biométrique côté serveur                      |
| Détection des gestes (temps réel) | MediaPipe Face Mesh (Google)               | Liveness côté client                                     |
| Comparaison biométrique           | DeepFace (FaceNet512 / VGG-Face / ArcFace) | Extraction et comparaison d'embeddings                   |
| Base de données                   | PostgreSQL + extension pgvector            | Stockage et recherche des embeddings                     |
| Cache                             | SQLite local ou Redis                      | Réduction de la latence sur les vérifications fréquentes |
| Authentification développeurs     | Firebase Auth (Google, GitHub)             | Gestion des comptes du dashboard développeur             |

## Paradigme de sécurité hybride client/serveur

Le principe fondateur de l'architecture FaceAuth est de **séparer la détection de vivacité (côté client) de la comparaison biométrique (côté serveur)**, afin d'atteindre l'objectif de performance fixé à moins de 2 secondes par vérification, y compris dans des contextes de connectivité instable (réseaux mobiles africains notamment).

### Pourquoi cette séparation est nécessaire

Envoyer un flux vidéo continu — ou une succession d'images — au serveur pour valider chacun des quatre gestes de liveness ferait exploser la latence perçue, en particulier sur des connexions 3G/4G instables. La solution retenue consiste à :

1. Charger MediaPipe Face Mesh directement dans le navigateur (JavaScript).
2. Valider localement, instantanément, et sans aller-retour réseau, la réalisation des quatre gestes de sécurité.

3. Une fois le défi réussi, capturer **une seule image finale** haute qualité et l'envoyer au serveur FastAPI.
4. Le serveur se limite à l'extraction de l'embedding facial et à la comparaison vectorielle dans pgvector.

Cette approche garantit une latence de validation des gestes nulle côté serveur (0 ms réseau pour cette étape), tout le calcul étant déporté sur le processeur de l'appareil de l'utilisateur final.

## Les quatre gestes de sécurité (liveness)

Pour le MVP, le système sélectionne aléatoirement un ordre de validation parmi quatre gestes, chacun associé à une rétroaction visuelle progressive (bordure de l'écran qui devient verte) :

| Geste   | Action utilisateur                 | Rétroaction visuelle  |
|---------|------------------------------------|-----------------------|
| Geste 1 | Clignement des yeux                | Bordure haute → vert  |
| Geste 2 | Ouverture de la bouche             | Bordure droite → vert |
| Geste 3 | Rotation de la tête vers la gauche | Bordure basse → vert  |
| Geste 4 | Rotation de la tête vers la droite | Bordure gauche → vert |

L'ordre de ces gestes est généré aléatoirement côté serveur à chaque session de vérification, ce qui empêche la réutilisation d'enregistrements vidéo préexistants (voir section sur l'anti-spoofing). Une icône centrale animée via un fichier Lottie (.json) guide visuellement l'utilisateur dans la réalisation du geste demandé (fonctionnalité prévue en post-MVP).

## Optimisation du backend FastAPI et DeepFace

### Pré-chargement des modèles en mémoire

Le chargement à la volée d'un modèle DeepFace ajoute entre 4 et 6 secondes par requête — un délai incompatible avec l'objectif de performance. La solution consiste à utiliser l'événement *lifespan* de FastAPI pour charger le modèle (FaceNet512 ou ArcFace) en mémoire vive une fois, au démarrage du conteneur sur Railway, et à le conserver actif pour toutes les requêtes suivantes.

### Normalisation des images en amont

Le SDK redimensionne et compresse l'image côté client avant l'envoi. DeepFace n'a besoin que d'une résolution de 112x112 ou 224x224 pixels : cette normalisation réduit

drastiquement le poids du payload réseau, ce qui est particulièrement important sur les connexions mobiles à bande passante limitée.

## Gestion de la concurrence

L'extraction DeepFace est une opération CPU-bound (bloquante). Les routes de prédiction doivent être déclarées avec `def` standard plutôt que `async def` : FastAPI exécute alors automatiquement ces requêtes dans un pool de threads séparé, ce qui évite de geler l'application sur des requêtes longues.

## Optimisation de la base de données (PostgreSQL + pgvector)

### Indexation vectorielle HNSW

À mesure que la table d'embeddings grandit, une recherche séquentielle devient rapidement trop lente. L'indexation **HNSW (Hierarchical Navigable Small World)** est recommandée plutôt que IVFFlat, car elle offre un meilleur compromis vitesse/précision sur de grands volumes :

```
CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops);
```

### Stratégie de cache Redis

Les embeddings de référence des utilisateurs actifs sont mis en cache dans Redis, ce qui évite de solliciter PostgreSQL à chaque vérification récurrente (modification de profil, validation de transaction répétée, etc.).

## Infrastructure et scaling (Railway)

- **Profil d'instance recommandé** : minimum 2 vCPU et 4 Go de RAM, DeepFace étant gourmand en ressources CPU/RAM.
- **Autoscale horizontal** : configuration du déploiement pour cloner automatiquement l'instance FastAPI dès que l'utilisation CPU dépasse 70 %.

### Point d'attention : cold start

Sur Railway, si l'instance redémarre ou se met en veille pour inactivité (selon le plan tarifaire), le chargement initial du modèle DeepFace en mémoire peut introduire un délai significatif sur la première requête. Il est recommandé de :

- Choisir un plan garantissant une disponibilité permanente (pas de scale-to-zero) pour l'environnement de production.



- Mettre en place un mécanisme de “warm-up” (requête de santé périodique) si un plan avec mise en veille est utilisé en environnement de test.

## Workflow chronométré d'une vérification réussie

| Étape | Intervalle    | Description                                                    |
|-------|---------------|----------------------------------------------------------------|
| 1     | 0,0 s - 0,5 s | Envoi de l'image compressée (réseau 4G/Wi-Fi)                  |
| 2     | 0,5 s - 1,2 s | Extraction de l'embedding par DeepFace (CPU serveur)           |
| 3     | 1,2 s - 1,3 s | Comparaison de la distance cosinus via pgvector ou Redis       |
| 4     | 1,3 s - 1,4 s | Écriture des logs, déduction du crédit, destruction de l'image |
| 5     | 1,4 s - 1,6 s | Renvoi du JSON de succès au client                             |

**Temps total moyen estimé : ~1,6 seconde** (objectif de moins de 2 secondes atteint).

# Besoins fonctionnels et non fonctionnels

---

## Besoins fonctionnels

### Dashboard développeur

- Création de compte simplifiée (via Firebase Auth — Google, GitHub).
- Génération d'une clé API unique au format `sk_live_...`.
- Affichage de la jauge de crédits restants en temps réel.
- Interface d'achat de packs de crédits supplémentaires.

### Dashboard client / module de scan

- Flux caméra fluide avec "Ghost Overlay" (silhouette de guidage pour le positionnement du visage).
- Génération aléatoire de l'ordre des quatre gestes, déterminée côté serveur à chaque session.
- Rétroaction visuelle progressive (bordures de l'écran qui s'allument au fur et à mesure de la validation des gestes).
- Envoi de l'image finale chiffrée vers le serveur, extraction de l'embedding par DeepFace, suppression immédiate de l'image, puis comparaison par produit scalaire / similarité cosinus dans pgvector.
- Retour d'une réponse JSON normalisée ( `success` / `error` ).

## Besoins non fonctionnels

### Performance et latence

Temps de réponse global de l'API inférieur à 2 secondes, grâce au pré-chargement des modèles DeepFace en mémoire vive sur Railway.

### Sécurité des données

Zéro stockage de photos brutes à long terme. L'image source est détruite en moins de 500 ms après la génération de l'embedding vectoriel irréversible. Ce point est développé en détail dans la section consacrée à la conformité (voir Section 6).

## **Fiabilité — marge de sécurité**

Système de blocage automatique de l'API (réponse HTTP 402 — *Payment Required*) si le solde de crédits du développeur atteint zéro. Ce mécanisme protège FaceAuth contre une consommation de ressources non rémunérée.

## **Clarté UX**

Des messages d'erreur textuels courts s'affichent sous le module de scan en cas de problème, par exemple :

- “Environnement trop sombre”
- “Visage trop loin”

# Spécifications de l'interface utilisateur (UX)

---

## Principe directeur : zéro redirection

Pour garantir une friction minimale et éliminer tout sentiment de redirection, le SDK FaceAuth n'isole jamais l'utilisateur sur une nouvelle page. L'interface de scan s'intègre directement dans la page hôte via une structure responsive qui détecte automatiquement la taille de l'écran de l'utilisateur final.

## Comportement adaptatif (desktop vs mobile)

### Sur ordinateur (desktop) — modale centrée

- Apparition au centre de l'écran avec un effet de fondu fluide.
- Assombrissement de l'arrière-plan (effet *backdrop*).
- Design épuré isolant le module de scan sans surcharger la vue.

### Sur smartphone (mobile) — bottom-sheet immersive

- Volet qui glisse de manière fluide du bas vers le haut de l'écran (transition d'environ 300 ms).
- Occupe entre 75 % et 90 % de la hauteur de l'écran, afin de maximiser l'espace dédié à la caméra.
- Zone d'interaction optimisée pour le pouce (*thumb zone*), conformément aux standards natifs iOS et Android.

## Règles de sécurité UX et ergonomie

- **Verrouillage de contexte** : le balayage vers le bas (*swipe to close*) et le clic en dehors de la zone (*click outside*) sont désactivés pendant l'exécution active des quatre gestes, afin d'éviter toute fermeture accidentelle qui interromprait le processus de vérification.
- **Bouton de secours** : un bouton "Annuler" discret est positionné en haut à droite du module, permettant à l'utilisateur d'interrompre volontairement le processus.
- **Isolation CSS (Shadow DOM)** : le module FaceAuth est encapsulé via le Shadow DOM, garantissant que son style (thèmes clair/sombre, bordures progressives) ne soit jamais altéré par les feuilles de style globales du site hôte, et inversement.

## Comportement du thème

### Mode clair (détection automatique)

- Fond clair (blanc).
- Texte noir ou gris foncé.
- Bordures discrètes.
- Contraste élevé pour une bonne lisibilité face à la caméra.

### Mode sombre (détection automatique)

- Fond sombre (noir).
- Texte blanc ou gris clair.
- Bordures légèrement lumineuses.
- Contraste optimisé pour les environnements à faible luminosité.

### Fallback système

Si le thème du système de l'utilisateur n'est pas détectable, le SDK applique le **mode clair** par défaut.

# Approfondissement — Sécurité anti-spoofing

---

## Le problème : une liveness purement côté client est falsifiable

L'architecture hybride décrite en Section 2 — déporter la détection des quatre gestes côté client (navigateur) et n'envoyer qu'une image finale au serveur — est indispensable pour respecter la contrainte de latence. Cependant, elle introduit un **angle mort de sécurité** s'il n'est pas correctement traité : un attaquant techniquement compétent pourrait potentiellement contourner le SDK JavaScript et envoyer directement une image au point de terminaison de l'API, sans avoir réellement exécuté les quatre gestes de vivacité demandés.

Les vecteurs d'attaque typiques incluent :

- **Replay d'image statique** : présentation d'une photo imprimée ou affichée sur un écran face à la caméra.
- **Deepfake / vidéo synthétique** : utilisation d'un visage généré ou modifié en temps réel pour simuler les mouvements attendus.
- **Appel direct à l'API** : un attaquant ayant compromis ou inspecté le SDK JS pourrait appeler directement l'endpoint de comparaison, en contournant entièrement la validation des gestes.
- **Attaque "man-in-the-browser"** : interception et modification du payload entre le navigateur et le serveur.

## Recommandations pour le MVP

### 1. Jeton de session de vérification (Challenge Token)

Chaque session de vérification doit être initiée par un appel serveur qui génère :

- Un **identifiant de session unique** (UUID).
- Une **séquence aléatoire des quatre gestes** (déjà prévue dans la spécification).
- Un **jeton signé à durée de vie courte** (par exemple JWT signé, valide 60 à 90 secondes), incluant la séquence de gestes attendue.

Le SDK client ne peut soumettre l'image finale qu'accompagnée de ce jeton. Le serveur vérifie :

- La validité et la non-expiration du jeton.

- Que le jeton n’a jamais été utilisé auparavant (protection contre le *replay*, via un enregistrement temporaire en cache Redis avec TTL correspondant à la durée de vie du jeton).

Cela n’empêche pas à 100 % un contournement, mais élève significativement la barrière technique et empêche la réutilisation simple d’un même jeton/image.

## 2. Preuve d’exécution côté client (Attestation légère)

Le SDK peut transmettre, en plus de l’image, un **résumé signé localement** des événements de détection (timestamps de chaque geste validé, scores de confiance MediaPipe). Bien que cette attestation puisse théoriquement être falsifiée par un attaquant très avancé, elle :

- Décourage la majorité des tentatives de fraude automatisée à faible coût.
- Permet de détecter des incohérences temporelles (par exemple, quatre gestes validés en moins de 200 ms, ce qui est physiologiquement impossible).

## 3. Vérifications anti-spoofing serveur (passives)

Indépendamment de la détection de gestes, le serveur peut effectuer des contrôles passifs sur l’image finale reçue, sans coût de latence significatif :

- **Détection d’écran / impression** : analyse de motifs de moiré, de reflets et de bords rectangulaires caractéristiques d’une photo affichée sur un écran ou imprimée (les bibliothèques DeepFace et OpenCV permettent des heuristiques basiques ; des modèles dédiés “anti-spoofing” légers — ex. Silent-Face-Anti-Spoofing — peuvent être ajoutés en post-MVP).
- **Cohérence de qualité d’image** : vérification de la résolution, de la netteté et de l’absence d’artefacts de compression excessive incompatibles avec une capture caméra directe.
- **Limitation de débit (rate limiting)** : un même compte développeur, une même clé API, ou une même empreinte d’appareil ne peut initier qu’un nombre limité de sessions de vérification par minute, afin de freiner les tentatives automatisées à grande échelle.

## 4. Roadmap post-MVP

Pour les versions ultérieures, en particulier si des clients fintech à fort enjeu réglementaire sont visés, il est recommandé d’évaluer :

- L’intégration d’un modèle d’anti-spoofing dédié (détection de profondeur, analyse de texture de peau).
- Une attestation d’intégrité de l’appareil (Web Attestation API, ou équivalents mobiles type Play Integrity API / Apple DeviceCheck pour les futurs SDK natifs).

- Un score de risque agrégé combinant signaux comportementaux (vitesse de mouvement, micro-tremblements) plutôt qu'une simple validation binaire des quatre gestes.

## Synthèse

| Niveau de risque             | Mesure                                                           | Statut recommandé   |
|------------------------------|------------------------------------------------------------------|---------------------|
| Faible coût d'implémentation | Jeton de session signé à courte durée de vie + anti-replay Redis | MVP — prioritaire   |
| Faible coût d'implémentation | Rate limiting par clé API / empreinte d'appareil                 | MVP — prioritaire   |
| Coût modéré                  | Attestation locale des timestamps de gestes                      | MVP — recommandé    |
| Coût modéré à élevé          | Modèle d'anti-spoofing dédié (texture, profondeur)               | Post-MVP            |
| Coût élevé                   | Attestation d'intégrité de l'appareil (natif)                    | Avec les SDK natifs |

Il est important de communiquer de manière transparente auprès des clients fintech sur le **niveau de protection effectif du MVP**, afin d'éviter toute survente d'un niveau de sécurité qui ne serait pleinement atteint qu'avec les évolutions post-MVP.



# Approfondissement — Modèle économique et tarification

---

## Rappel du modèle actuel

FaceAuth repose sur un modèle prépayé de type *pay-as-you-go*, où chaque vérification biométrique consomme exactement un crédit. L'argent est encaissé avant la consommation des ressources serveur, ce qui élimine le risque d'impayés liés à l'usage.

| Pack       | Volume         | Prix                                     | Prix unitaire    |
|------------|----------------|------------------------------------------|------------------|
| Découverte | 500 crédits    | Gratuit (offert à la création du compte) | —                |
| Croissance | 2 000 crédits  | 100 \$                                   | 0,05 \$ / crédit |
| Échelle    | 10 000 crédits | 400 \$                                   | 0,04 \$ / crédit |
| Entreprise | Sur mesure     | Sur devis                                | À négocier       |

## Analyse de la structure tarifaire

### Cohérence de la dégressivité

La dégressivité du prix par crédit (0,05 \$ puis 0,04 \$) est cohérente avec les pratiques standard du secteur : elle récompense le volume tout en incitant les clients à passer au palier supérieur. L'écart de 20 % entre les deux paliers payants reste toutefois modeste — il pourrait être intéressant d'introduire un palier intermédiaire (par exemple 5 000 crédits) pour fluidifier la progression entre “Croissance” et “Échelle”, et augmenter le taux de conversion vers des paniers plus importants.

### Estimation des coûts d'infrastructure par vérification

Sur la base du profil d'instance recommandé (2 vCPU / 4 Go de RAM sur Railway), il est essentiel d'estimer le coût réel par vérification afin de valider la marge :

- Le coût d'une instance Railway de ce profil se situe, à titre indicatif, dans une fourchette de quelques dizaines à une centaine de dollars par mois selon le plan et le taux d'utilisation.
- En supposant qu'une instance traite en continu plusieurs vérifications par seconde en heures de pointe, le coût marginal par vérification (hors coûts fixes

de base de données et de stockage) reste typiquement de l'ordre de quelques fractions de centime à 0,01 \$.

- À 0,04 \$ - 0,05 \$ par crédit, la marge brute apparente est confortable **à condition que le taux d'utilisation des instances reste élevé** (l'autoscaling et le pré-chargement des modèles évitent le gaspillage de ressources sur des instances sous-utilisées).

**Recommandation** : mettre en place, dès le lancement, un suivi des coûts d'infrastructure par vérification réelle (et non théorique), afin d'ajuster la grille tarifaire si nécessaire avant une croissance importante du volume.

## **Le pack “Découverte” gratuit — opportunités et risques**

Offrir 500 crédits gratuits à la création de compte est un excellent levier d'acquisition pour des développeurs en phase de test ou de prototypage. Cependant, ce modèle est exposé à un risque d'abus par création de comptes multiples (*multi-accounting*) pour cumuler des crédits gratuits sans jamais payer.

### **Mesures de mitigation recommandées :**

- Lier la création de compte développeur à une vérification d'identité légère via Firebase Auth (Google/GitHub), qui réduit déjà partiellement la facilité de multi-comptes.
- Envisager de demander une carte bancaire valide (même sans prélèvement à 0 \$) pour débloquent le pack gratuit complet — pratique courante chez les fournisseurs d'API (Twilio, SendGrid, etc.).
- Limiter le pack gratuit par domaine e-mail vérifié et/ou par numéro de téléphone, plutôt que par simple adresse e-mail.
- Surveiller les patterns d'usage anormaux (comptes créés en rafale depuis la même IP, consommation immédiate des 500 crédits suivie d'inactivité).

## **Positionnement concurrentiel**

Le marché de la vérification d'identité biométrique (KYC-as-a-Service) compte des acteurs établis dont les tarifs de référence se situent généralement entre 0,50 \$ et 2 \$ ou plus par vérification complète (incluant souvent vérification de documents, contrôle de listes de sanctions, etc.). FaceAuth, en se concentrant uniquement sur la biométrie faciale et la détection de vivacité (sans vérification documentaire ni screening réglementaire), se positionne sur un segment plus restreint et donc plus économique — ce qui en fait un complément potentiel à des solutions KYC complètes plutôt qu'un concurrent direct frontal sur l'ensemble du parcours KYC.

**Recommandation stratégique** : positionner clairement FaceAuth comme une **bricke de liveness et de ré-authentification biométrique pour actions sensibles post-onboarding**, en complément d'une solution KYC initiale (vérification

de documents d'identité), plutôt que comme une solution KYC complète à elle seule — sauf si une roadmap d'extension vers la vérification documentaire est envisagée.

## **Pack Entreprise**

Le pack “Entreprise sur commande” est adapté pour les grands comptes fintech. Il est recommandé d'y associer, dès que possible :

- Des engagements de niveau de service (SLA) formalisés (disponibilité, temps de réponse garanti).
- Des clauses contractuelles spécifiques à la conformité (voir Section 6), notamment concernant la localisation des données et les registres d'audit.
- Une tarification pouvant inclure un engagement de volume minimum mensuel, plus adapté aux besoins de prévisibilité budgétaire des grandes organisations.

# Conformité réglementaire et protection des données (KYC)

---

## Contexte

FaceAuth traite des **données biométriques**, qui sont juridiquement qualifiées de données à caractère personnel **sensibles** dans la quasi-totalité des cadres réglementaires (RGPD européen, lois de protection des données de nombreux pays africains inspirées de la Convention de Malabo de l'Union Africaine, CCPA/CPRA aux États-Unis, etc.). Le traitement de ce type de données impose des obligations renforcées, et ce, indépendamment du fait que FaceAuth agisse comme sous-traitant technique pour ses clients développeurs.

Cette section présente un panorama des principes de conformité à intégrer dans la conception de FaceAuth. **Elle constitue un point de départ et ne remplace pas une consultation juridique formelle**, en particulier avant tout déploiement commercial dans une juridiction donnée.

## Principes déjà intégrés dans la conception technique

L'architecture FaceAuth intègre déjà plusieurs principes alignés avec les exigences réglementaires courantes :

- **Minimisation des données** : seule une image finale est transmise au serveur, et uniquement après normalisation (réduction de résolution).
- **Limitation de la conservation** : l'image source est détruite en moins de 500 ms après extraction de l'embedding, ce qui correspond au principe de *storage limitation* du RGPD (article 5.1.e) et d'équivalents dans d'autres cadres.
- **Sécurité par conception** : chiffrement de l'image en transit, isolation des composants, suppression immédiate des données brutes.
- **Irréversibilité de l'embedding** : la donnée conservée (le vecteur d'embedding) est présentée comme irréversible, ce qui réduit (sans l'annuler totalement) le risque en cas de fuite de la base de données.

## Points à formaliser pour une documentation de conformité robuste

### 1. Statut juridique de FaceAuth (sous-traitant vs responsable de traitement)

Dans la plupart des cadres (RGPD notamment), FaceAuth agira très probablement en tant que **sous-traitant** (processor) pour le compte de ses clients développeurs, qui restent **responsables de traitement** (controllers) vis-à-vis des utilisateurs finaux. Cette qualification doit être :

- Explicitement formalisée dans les conditions générales d'utilisation (CGU) et dans un **Accord de traitement des données (Data Processing Agreement — DPA)** signé avec chaque client.
- Cohérente avec les responsabilités effectivement assumées : FaceAuth détermine les moyens techniques (comment les données sont traitées), tandis que le client développeur détermine les finalités (pourquoi il utilise FaceAuth).

### 2. Base légale du traitement

Le client développeur (responsable de traitement) doit disposer d'une base légale pour soumettre les données biométriques de ses utilisateurs à FaceAuth — typiquement le **consentement explicite** de l'utilisateur final, ou dans certains cas un **intérêt légitime** encadré (lutte contre la fraude) selon la juridiction. FaceAuth devrait :

- Fournir une documentation type ("mentions légales modèles") que ses clients peuvent adapter pour informer leurs utilisateurs finaux.
- Ne jamais présumer que le consentement a été recueilli — cette responsabilité incombe contractuellement au client développeur, mais FaceAuth a intérêt à l'exiger explicitement dans ses CGU pour limiter sa propre exposition.

### 3. Registre des traitements et durée de conservation des embeddings

Si l'image brute est détruite en moins de 500 ms, **l'embedding vectoriel, lui, est conservé** (c'est la donnée de référence permettant les comparaisons futures). Il convient de documenter formellement :

- La **durée de conservation** des embeddings (par exemple : durée de la relation contractuelle entre l'utilisateur final et le client développeur, plus une durée de purge définie).
- Le **droit à l'effacement** : un mécanisme permettant à un client développeur (sur demande de son utilisateur final, exerçant son droit à l'oubli) de supprimer définitivement l'embedding associé à un utilisateur donné.
- La **portabilité** : dans certains cadres, possibilité pour un utilisateur de demander la suppression de son embedding sans réinscription complète.

## 4. Localisation des données et transferts internationaux

Pour des clients fintech opérant dans des juridictions africaines avec des exigences de **localisation des données** (certaines réglementations bancaires centrales imposent que les données des clients soient stockées sur le territoire national ou régional), il est essentiel de documenter :

- La localisation géographique effective des serveurs PostgreSQL/pgvector et de l'infrastructure Railway.
- Les mécanismes de transfert de données le cas échéant (clauses contractuelles types, décisions d'adéquation), si des données transitent entre régions (par exemple entre l'Afrique et l'Europe ou les États-Unis selon la région d'hébergement Railway choisie).

**Recommandation** : pour les clients fintech soumis à des exigences de localisation strictes, prévoir une option de déploiement régional (instance Railway et base de données dans la même région géographique que les opérations du client), ou à défaut documenter précisément les flux transfrontaliers existants.

## 5. Journalisation et auditabilité

Les obligations de conformité KYC imposent souvent une **traçabilité forte** des opérations de vérification. FaceAuth doit prévoir :

- Un **journal d'audit immuable** de chaque vérification : horodatage, identifiant de session, résultat (succès/échec), score de similarité (sans stocker l'image), identifiant du crédit consommé.
- Une preuve documentée du respect du délai de destruction de l'image (par exemple, log technique avec horodatage de réception et horodatage de suppression).
- Un accès pour le client développeur à ses propres journaux d'audit, pour répondre lui-même à ses propres obligations réglementaires vis-à-vis de ses régulateurs.

## 6. Sécurité technique attendue

En complément des mesures déjà prévues (chiffrement en transit, suppression immédiate de l'image), il est recommandé de documenter et formaliser :

- Le chiffrement au repos de la base de données (embeddings).
- La gestion des clés API ( `sk_live_...` ) : rotation possible, révocation immédiate en cas de compromission, scopes limités.
- Une politique de gestion des incidents de sécurité, incluant les délais de notification aux clients développeurs (et, en cascade, aux autorités compétentes) en cas de violation de données — le RGPD impose par exemple une notification à l'autorité de contrôle dans un délai de 72 heures.

## 7. Biais algorithmique et équité

Les modèles de reconnaissance faciale (FaceNet, VGG-Face, ArcFace) peuvent présenter des **biais de performance différentielle** selon les caractéristiques démographiques (teint de peau, genre, âge). Pour un produit visant explicitement un déploiement en Afrique, il est recommandé de :

- Documenter les performances du modèle retenu sur des jeux de données représentatifs de la diversité des utilisateurs finaux visés.
- Prévoir un mécanisme de recours humain en cas de rejet répété (échec systématique de la vérification), afin d'éviter qu'un utilisateur légitime ne soit durablement bloqué dans l'accès à un service financier — point sensible tant sur le plan éthique que sur celui de la conformité (non-discrimination).

### Tableau de synthèse — feuille de route conformité

| Domaine                         | Action recommandée                                                                     | Priorité                                   |
|---------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------|
| Qualification juridique         | Rédiger un DPA type (FaceAuth = sous-traitant)                                         | MVP / avant commercialisation              |
| Documentation utilisateur final | Fournir des mentions légales modèles pour les clients développeurs                     | MVP / avant commercialisation              |
| Conservation des données        | Définir et documenter la durée de conservation des embeddings + mécanisme d'effacement | MVP                                        |
| Journalisation                  | Mettre en place un journal d'audit immuable des vérifications                          | MVP                                        |
| Localisation des données        | Documenter la région d'hébergement ; envisager des options régionales                  | Avant clients fintech à exigences strictes |
| Sécurité                        | Chiffrement au repos, gestion des clés API, politique d'incident                       | MVP                                        |
| Équité algorithmique            | Évaluer les performances du modèle sur une population diversifiée                      | Avant déploiement à grande échelle         |

## Avertissement

Cette section présente un cadre d'analyse général destiné à structurer la réflexion sur la conformité. Les obligations réglementaires précises varient significativement selon les pays d'opération des clients de FaceAuth (chaque pays africain, ainsi que l'Union Européenne et d'autres juridictions, disposant de leurs propres textes en matière de protection des données personnelles et de réglementation bancaire). **Il est fortement recommandé de faire valider cette documentation par un conseil juridique spécialisé en protection des données et en réglementation financière**, avant toute commercialisation auprès de clients fintech.



# Synthèse et prochaines étapes

---

## Récapitulatif de faisabilité

Le projet FaceAuth est **techniquement faisable** avec la stack retenue, et l'architecture hybride client/serveur constitue une réponse pertinente à la contrainte de latence dans des contextes de connectivité variable. Les principaux axes de vigilance pour passer du MVP à un produit commercialisable auprès de clients fintech sont :

1. **Renforcer la sécurité anti-spoofing** au-delà de la simple validation de gestes côté client, via un système de jetons de session et des contrôles passifs côté serveur.
2. **Valider la robustesse sur appareils bas de gamme**, particulièrement pertinents pour le marché africain visé.
3. **Formaliser la documentation de conformité** (DPA, durées de conservation, journalisation d'audit) avant toute commercialisation auprès de clients soumis à des obligations KYC.
4. **Affiner la grille tarifaire** sur la base de coûts d'infrastructure réels mesurés en conditions de production.

## Prochaines étapes suggérées

- Développement d'un prototype fonctionnel du module de scan (SDK JS) avec MediaPipe Face Mesh et les quatre gestes de liveness.
- Mise en place de l'environnement FastAPI avec pré-chargement DeepFace sur Railway, et mesure réelle des temps de réponse en charge.
- Rédaction du DPA type et des mentions légales modèles, en lien avec un conseil juridique.
- Tests de performance sur une sélection représentative d'appareils mobiles d'entrée de gamme.

---

*Document préparé à des fins de documentation interne et de discussion avec les parties prenantes du projet FaceAuth. Les éléments relatifs à la conformité réglementaire ne constituent pas un avis juridique.*